# Diseño de algoritmos (2125)

Parcial 1: Unidades 1 a 3 · vale 17% · 1 h 5 min · 1 intento · Basado en el apunte digital FCA-UNAM (plan 2024)

> Es teórico, con algo de evaluación de expresiones. Lo más preguntable: características del algoritmo, partes de la máquina de Turing, fases del ciclo de vida, tipos de mantenimiento, intérprete y compilador, fases de compilación, tipos de datos, operadores y **precedencia**.

## Unidad 1 · Algoritmos, autómatas y máquina de Turing

- **Algoritmo:** conjunto detallado de pasos **finitos y lógicos** para resolver un problema. Etimología: del árabe, cálculo con cifras arábigas (María Moliner). A la computadora hay que indicarle **cada paso**, "como a un niño pequeño" (ejemplos del apunte: intercambiar 24 y 9 con una variable auxiliar; agua de limón).
- **Tipos: cualitativos** = instrucciones en texto **sin cálculos matemáticos** (cambiar una llanta). **Cuantitativos** = **con cálculos matemáticos**.
- **Partes: entrada → proceso → salida**.
- **Características:** una entrada (**cero o más** datos), un proceso, una salida (**uno o más** resultados), **finito** (debe terminar), **ordenado**, **claro** y **confiable** (siempre produce el mismo resultado).

### Autómatas y lenguajes formales

- **Autómata:** modelo computacional con un **conjunto de estados**, un **estado inicial**, un **alfabeto de entrada** y una **función de transición**. Equivale a autómata finito o máquina de estados finitos. **Estado** = su condición en un instante.
- Historia: cajas de música (s. XVII), la Tañedora de Salterio (Roentgen y Kintzing), **el Dibujante y los Músicos** (Pierre Jaquet-Droz), el **Pato con aparato digestivo** (Vaucanson), el telar de Jacquard y la máquina de Falcon. Ejemplos actuales: **termostato** y **elevador**.
- Se representan con **diagramas de estado** (círculos = estados; flechas = transiciones con entrada/salida) y **tablas de estado** (estado actual, entrada, estado siguiente y salida).
- **Lenguaje formal:** conjunto de símbolos y reglas; se forma con **alfabeto, vocabulario y gramática**. **Alfabeto** = conjunto de todos los **símbolos válidos** (puede ser binario: 0 y 1). **Gramática**: **define y enumera las palabras y frases válidas** de un lenguaje.
- **BNF** (Backus-Naur): describe la sintaxis. Regla de producción: `<regla> = sentencias y frases.` Elementos obligatorios: lado izquierdo entre < >, operador **=** y **punto final**. **|** = opciones excluyentes (OR); **?** = elemento opcional. **Símbolo terminal** = palabra del vocabulario (ARCHIVO). **No terminal** = regla (<opcion>).

### Máquina de Turing (Alan Turing, 1936)

| Elemento | Qué es |
|---|---|
| **Cinta** | **Infinita** (sin límite en los extremos), dividida en segmentos iguales con un símbolo o vacíos; es el **dispositivo de almacenamiento** |
| **Cabeza de lectura-escritura** | **Lee y escribe** en la cinta; actúa en un segmento y hace **una operación a la vez**; tiene un número finito de estados |
| **Programa** | Instrucciones que controlan los **movimientos de la cabeza** (a dónde moverse, leer o escribir) |

Es un autómata (finito determinista) que nunca se construyó. Sirve para definir qué puede y qué no puede hacer una computadora. Problemas **computables** = se pueden resolver con ella. **No computables o indecidibles** = **no tienen solución para todas sus entradas**. Notación: δ(q1, 0) = (q1, 1, D) significa "en el estado q1, si lee 0, queda en q1, escribe 1 y se mueve a la Derecha".

## Unidad 2 · Fundamentos de programación

**Programa** = ejecución o implementación de un algoritmo en una computadora. El algoritmo resuelve el problema **en teoría**; el programa, **en la práctica**. Hay problemas sin solución algorítmica (predecir un terremoto o una pandemia).

### Ciclo de vida de un programa (en orden)

| Fase | Qué se hace |
|---|---|
| 1. **Definición** | **Primer paso**: describir y **acotar** el problema (datos de entrada y resultado) |
| 2. **Análisis** | **Recabar la información** necesaria; decidir si se puede resolver con un programa. Técnicas: **entrevistas**, cuestionarios y observación. Si la solución es urgente se elige la **más rápida**; si hay pocos recursos, la **más económica**. |
| 3. **Diseño** | Representar la solución en **pseudocódigo** o diagrama de flujo (INICIO/FIN, palabras reservadas en MAYÚSCULAS, sangría, LEER y ESCRIBIR) |
| 4. **Codificación** | Escribir el **código fuente** en un lenguaje (C, C++, Java, PHP, Python), compilarlo o interpretarlo y probarlo. IDE: Eclipse, Dev-C++, NetBeans. |
| 5. **Pruebas y depuración** | **Depuración (debugging)** = identificar y corregir errores (de la polilla en una computadora de bulbos). Errores de **sintaxis** (más fáciles) y **lógicos** (el programa corre pero hace mal la tarea; más difíciles) |
| 6. **Documentación** | Nombre del programa y del programador, descripción, variables y constantes, fechas, entradas, salidas y mensajes de ejecución y de error |
| 7. **Implantación** | Instalar la versión final, adecuar hardware y software, **capacitar** a los usuarios y entregar manuales |
| 8. **Mantenimiento** | **Preventivo** (sin errores: respaldos, **verificar la integridad de los datos**) · **Correctivo** (cuando **hay errores**) · **Adaptativo** (nueva versión del SO o nuevo equipo) |

### Conceptos básicos

- **Programación:** todo el proceso de construir programas. **Lenguaje de programación:** representa algoritmos de forma inteligible para la computadora; la computadora solo entiende **lenguaje máquina** (0 y 1).
- **Librería:** conjunto de funciones (`#include <stdio.h>` = entrada y salida). **Palabra reservada:** tiene un significado especial y no puede usarse como identificador. **Variable:** accede a la memoria para guardar y recuperar datos; su valor **cambia**. **Constante:** su valor **no cambia**. **Tipo de dato:** clase de dato (entero, carácter, flotante). Operadores, comentarios y sentencias.
- **Traductor:** convierte el código fuente a lenguaje máquina. **Intérprete:** ejecuta **línea por línea**; fácil de modificar, pero más lento (**Perl, PHP, Python, JavaScript, Ruby**). **Compilador:** transforma el código fuente en **ejecutable** optimizado; más rápido (**C, C++, Pascal, Delphi, Fortran**).

### Fases de un compilador

| Fase | Función |
|---|---|
| 1. Analizador **léxico** | Lee carácter por carácter y genera **tokens** |
| 2. Análisis **sintáctico** | Comprueba el orden de los tokens y arma el **árbol sintáctico** |
| 3. Análisis **semántico** | Comprueba el **significado**; arma el árbol semántico |
| 4. Código **intermedio** | Para una máquina abstracta; más fácil de traducir |
| 5. Código **final** | Pasa a ensamblador y luego a ejecutable |
| 6. **Tabla de símbolos** | No es una fase: estructura tipo **diccionario** (inserción, borrado y búsqueda) con variables y funciones |
| 7. **Gestor de errores** | Informa de errores (sobre todo en las fases sintáctica y semántica) |

## Unidad 3 · Elementos básicos de un programa

- **Elementos:** inicio y fin, declaración de variables, constantes, instrucciones y lectura, proceso y salida de datos.
- **Estructuras: secuencial** (ejecuta todas las instrucciones **sin omitir ninguna**), **selectiva** (bifurca con **Si…De lo contrario**) e **iterativa** (repite **mientras se cumpla una condición**).
- Partes de un programa en C: bibliotecas (#include), función principal `main()`, llaves { } de inicio y fin, declaración de variables y desarrollo (printf, scanf, getch).
- **Identificador:** nombre que da el **programador** a los objetos (el nombre de una variable es un identificador). **Palabra reservada:** predefinida, no se usa para otro fin (en C: int, float, printf…; en pseudocódigo: INICIO y FIN).
- **Memoria principal (RAM):** temporal, rápida y cara. **Secundaria** (disco duro): más lenta. **Memoria virtual:** usar la secundaria cuando la RAM no alcanza (más lento). RAM ≠ disco duro.

| Elemento | Definición | Ejemplo |
|---|---|---|
| **Constante** | Valor que **nunca cambia** durante la ejecución | `REAL PI <- 3.141592;` `#define` |
| **Variable** | Unidad básica de almacenamiento (identificador + tipo); su valor **cambia** | `int numero1 = 1;` |
| **Arreglo** | Conjunto **ordenado** de elementos **homogéneos** (mismo tipo). Unidimensional = **vector**; bidimensional = **matriz** [renglón] [columna]. En C se cuenta desde 0. | `int x[100];` |
| **Registro** | Dato estructurado con **campos** del mismo tipo o de **distintos tipos** (heterogéneos); en C se llama **struct** | `struct coche {…};` |

### Tipos de datos

| Categoría | Tipos |
|---|---|
| **Numéricos** | **Enteros** (sin fracción; 8, 16 o 32 bits; int −32 768 a 32 767; unsigned int 0 a 65 535; también llamados de "punto fijo") y **reales o flotantes** (con punto decimal: float, double, long double) |
| **No numéricos** | **Caracteres** (un solo carácter: alfabéticos, numéricos o especiales; char −128 a 127; **cadena** = sucesión de caracteres entre comillas) y **booleanos** (solo **verdadero o falso**; C no los trae, se definen con typedef) |

**Declaraciones:** parte donde se definen las **constantes y variables** (en C es obligatorio; en PHP no). **Expresión:** combinación de constantes y variables (operandos) con operadores que representa un valor.

| Operadores | Símbolos |
|---|---|
| **Aritméticos** | + − * / y **% (módulo = residuo)**. En C la división de enteros desecha la fracción: 1/2 = 0, 3/2 = 1, −7/3 = −2. 12 % 5 = 2 y 12 % 3 = 0. |
| **Relacionales** | < > <= >= == (igual) != o <> (distinto); dan verdadero o falso |
| **Lógicos** | **AND** (V si ambos son V) · **OR** (V si alguno es V) · **NOT** (invierte) |
| **Asignación** | **<-** en pseudocódigo; **=** en C (x = y asigna el valor de y a x) |
| **Asociativo** | El único es el **paréntesis ( )** |

**Literales:** valores constantes. Numéricas (PI <- 3.1416), de **carácter** (entre comillas sencillas: 'a'), de **cadena** (entre comillas dobles: "hola") y **booleanas** (true/false).

> **Precedencia de operadores:** 1) ( ) paréntesis, del más interno hacia afuera · 2) ^ potencia · 3) * / mod · 4) + −. Con igual prioridad se evalúa de **izquierda a derecha**.
> 4 + 2 * 5 = **14** · (4 + 2) * 5 = **30** · 23 * 2 / 5 = **9.2** · 3 + 5 * (10 − (2 + 4)) = **23** · 2.1 * (1.5 + 3.0 * 4.1) = **28.98**

# Autoevaluaciones del apunte (clave oficial)

## Unidad 1

| Verdadero o falso | V/F |
|---|---|
| 1. Un algoritmo debe obtener siempre el mismo resultado. | **V** |
| 2. Un algoritmo puede saltarse pasos y aun así dar un resultado correcto. | **F** |
| 3. Un algoritmo se representa con un diagrama entidad-relación. | **F** |
| 4. Un algoritmo cualitativo contempla cálculos matemáticos. | **F** |
| 5. Un algoritmo puede representar secuencias infinitas. | **F** |
| 6. Los procesos computables y no computables pueden implementarse en una máquina de Turing. | **F** |
| 7. La cabeza de lectura-escritura lee y escribe en la cinta, una operación a la vez. | **V** |
| 8. La cinta de la máquina de Turing es necesariamente finita. | **F** |
| 9. El programa controla los movimientos de la cabeza de lectura-escritura. | **V** |
| 10. La cinta es solo de lectura y no de almacenamiento. | **F** |

| Relaciona / opción múltiple | Respuesta oficial |
|---|---|
| Conjunto de todos los símbolos válidos | Alfabeto |
| Conjunto de pasos finitos y lógicos para resolver un problema | Algoritmo |
| La máquina de Turing es ejemplo de un… | Autómata |
| Algoritmo sin cálculos / con cálculos | Cualitativo / Cuantitativo |
| Es una característica de un algoritmo | Es finito |
| Definir y enumerar las palabras y frases válidas de un lenguaje | La gramática |
| Autómata formado por cinta, cabeza y programa | Máquina de Turing |
| Problema que no puede implementarse en una máquina de Turing por no tener solución para todas sus entradas | Indecidible |
| Modelo con estados, estado inicial, alfabeto de entrada y función de transición ⚠ | Clave: "Función". Lo correcto según el texto: **Autómata** |

## Unidad 2

| Verdadero o falso | V/F |
|---|---|
| 1. El lenguaje C utiliza un compilador. | **V** |
| 2. PHP utiliza un compilador. | **F** (intérprete) |
| 3. Una entrevista es una forma de obtener información en la fase de diseño. | **F** (es en el análisis) |
| 4. Una librería contiene funciones de un lenguaje. | **V** |
| 5. Debugging es sinónimo de depuración. | **V** |
| 6. El valor de las variables es constante. | **F** |
| 7. El signo "+" es un tipo de dato. | **F** (es un operador) |
| 8. Una palabra reservada no puede usarse como identificador. | **V** |
| 9. Los errores lógicos se deben a fallas de sintaxis. | **F** |

| | |
|---|---|
| 10. El lenguaje C es un ejemplo de lenguaje máquina. | **F** |

| Relaciona / opción múltiple | Respuesta oficial |
|---|---|
| Identificar y corregir errores de programación | Depuración |
| El primer paso para resolver un problema es su… | Definición |
| Recabar la información necesaria para resolver un problema | Análisis |
| Mantenimiento que verifica la integridad de los datos | Preventivo |
| Mantenimiento cuando el programa presenta un error | Correctivo |
| Estructura tipo diccionario con inserción, borrado y búsqueda | Tabla de símbolos |
| Fase del ciclo de vida donde se hacen adecuaciones al programa | Mantenimiento |
| Lee línea por línea el código fuente (lo usa Perl) | Intérprete |
| Transforma el código fuente en ejecutable (lo usa C++) | Compilador |
| <stdio.h> es un ejemplo de | Librería |

### Unidad 3

| Verdadero o falso | V/F |
|---|---|
| 1. La secuencial ejecuta todas las sentencias sin omitir ninguna. | **V** |
| 2. La selectiva usa ciclos con un número determinado de iteraciones. | **F** |
| 3. La iterativa ejecuta una sentencia mientras se cumpla una condición. | **V** |
| 4. Una palabra reservada es un texto predefinido que no se puede usar para otro fin. | **V** |
| 5. INICIO es una palabra reservada del lenguaje C. | **F** (es de pseudocódigo) |
| 6. El nombre de una variable también es un identificador. | **V** |
| 7. La memoria principal es lo mismo que el disco duro. | **F** |
| 8. Las constantes nunca cambian durante la ejecución. | **V** |
| 9. Las variables nunca cambian durante la ejecución. | **F** |
| 10. Un arreglo es un conjunto de elementos heterogéneos. | **F** (homogéneos) |

| Relaciona / opción múltiple | Respuesta oficial |
|---|---|
| Conjunto de elementos heterogéneos (de distinto tipo) | Registro |
| Arreglo unidimensional / bidimensional | Vector / Matriz |
| Los numéricos comprenden los ___ y los de punto flotante | Enteros |
| Los no numéricos comprenden ___ y booleanos | Caracteres |
| Parte del algoritmo donde se definen constantes y variables | Declaraciones |
| La combinación de constantes y variables forma una | Expresión |
| Ejemplo de operador relacional | > |
| Una literal es un valor | Constante |
| Operador con mayor precedencia | ( ) |

## Práctica: 25 preguntas

**1.** Las tres partes de un algoritmo son:
a) análisis, diseño y prueba · b) inicio, medio y fin · c) entrada, proceso y salida · d) datos, código y resultado

**2.** Un algoritmo que no implica cálculos matemáticos es:
a) iterativo · b) cuantitativo · c) indecidible · d) cualitativo

**3.** Un termostato es un ejemplo de:
a) librería · b) compilador · c) gramática · d) autómata

**4.** En la notación BNF, el símbolo | significa:
a) asignación · b) opciones mutuamente excluyentes · c) fin de la regla · d) elemento opcional

**5.** La cinta de la máquina de Turing es:
a) un alfabeto · b) infinita y sirve como almacenamiento · c) finita y de solo lectura · d) parte del programa

**6.** δ(q1, 0) = (q1, 1, D) significa que en q1, al leer 0:
a) borra la cinta · b) escribe 0 y se detiene · c) cambia a q2 · d) escribe 1 y se mueve a la derecha

**7.** Un problema sin solución para todas sus entradas posibles es:
a) decidible · b) cualitativo · c) computable · d) indecidible

**8.** El orden correcto del ciclo de vida es:
a) análisis, definición, diseño… · b) codificación, diseño, análisis… · c) definición, análisis, diseño, codificación, pruebas, documentación, implantación, mantenimiento · d) diseño, análisis, pruebas…

**9.** Las entrevistas y cuestionarios se usan en la fase de:
a) diseño · b) codificación · c) implantación · d) análisis

**10.** La capacitación de los usuarios ocurre en la fase de:
a) documentación · b) mantenimiento · c) análisis · d) implantación

**11.** Adaptar un programa a una nueva versión del sistema operativo es mantenimiento:
a) preventivo · b) lógico · c) correctivo · d) adaptativo

**12.** El programa corre pero calcula mal: es un error:
a) de compilación · b) léxico · c) lógico · d) de sintaxis

**13.** ¿Cuál de estos lenguajes usa un intérprete?
a) Pascal · b) C · c) Python · d) Fortran

**14.** La fase del compilador que genera los tokens es:
a) léxica · b) sintáctica · c) semántica · d) código final

**15.** La fase que arma el árbol sintáctico y revisa el orden de los tokens es:
a) semántica · b) léxica · c) gestor de errores · d) sintáctica

**16.** Usar el disco duro cuando la RAM no alcanza se llama:
a) memoria caché · b) registro · c) memoria principal · d) memoria virtual

**17.** Un arreglo está formado por elementos:
a) heterogéneos · b) constantes · c) homogéneos · d) booleanos

**18.** El tipo de dato que solo toma verdadero o falso es:
a) carácter · b) booleano · c) cadena · d) flotante

**19.** En C, ¿cuánto vale 7/2 si ambos son enteros? ¿Y 17 % 5?
a) 4 y 2 · b) 3 y 2 · c) 3.5 y 3 · d) 3 y 3

**20.** 10 − 2 * 3 + 4 =
a) 12 · b) 28 · c) 0 · d) 8

**21.** (10 − 2) * (3 + 4) =
a) 56 · b) 17 · c) 32 · d) 28

**22.** 20 / 4 * 2 =
a) 40 · b) 5 · c) 2.5 · d) 10

**23.** Si A = V y B = F: A AND B, A OR B, NOT A dan:
a) F, F, V · b) V, V, F · c) F, V, F · d) V, F, V

**24.** "hola" entre comillas dobles es una literal:
a) de carácter · b) de cadena · c) numérica · d) booleana

**25.** La estructura que repite instrucciones mientras se cumple una condición es:
a) secuencial · b) iterativa · c) declarativa · d) selectiva

**Clave:** 1-c · 2-d · 3-d · 4-b · 5-b · 6-d · 7-d · 8-c · 9-d · 10-d · 11-d · 12-c · 13-c · 14-a · 15-d · 16-d · 17-c · 18-b · 19-b · 20-d · 21-a · 22-d · 23-c · 24-b · 25-b